ROS basics

Connect the pieces of a robot.

Understand ROS **Read the code** **Inspect it live**



What is ROS?

A shared framework for building robotics software.

Robot Operating System

Connect software across languages and machines. Share common interfaces.

Robot application

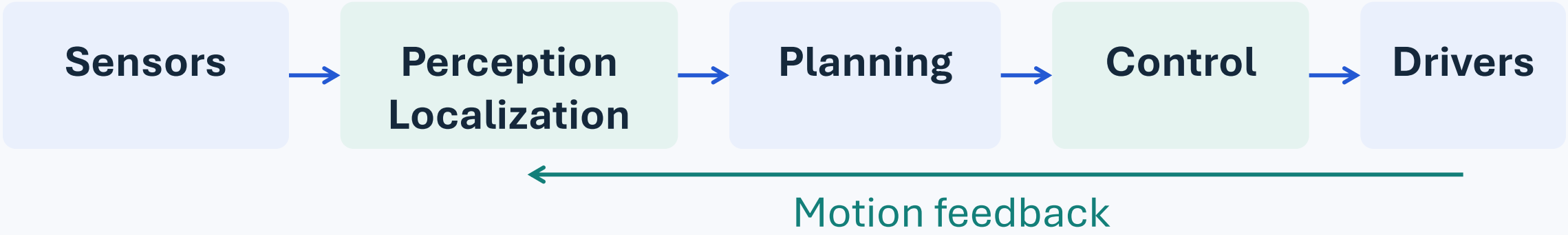
ROS libraries + tools + packages

Host OS + hardware

Reuse drivers and algorithms. Focus on your robot's behavior.

What is ROS used for?

Integrate sensing, decisions, and actuation in one system.



LOGISTICS

BMW + Microsoft / interoperability

MANUFACTURING

Foundries / scan → plan → blend

ROS 1 → ROS 2

Familiar concepts. A redesigned foundation.

2007 ROS begins

2017 ROS 2 Ardent

2024 Jazzy

2025 ROS 1 EOL

ROS 1

Master discovery · TCPROS

ROS 2

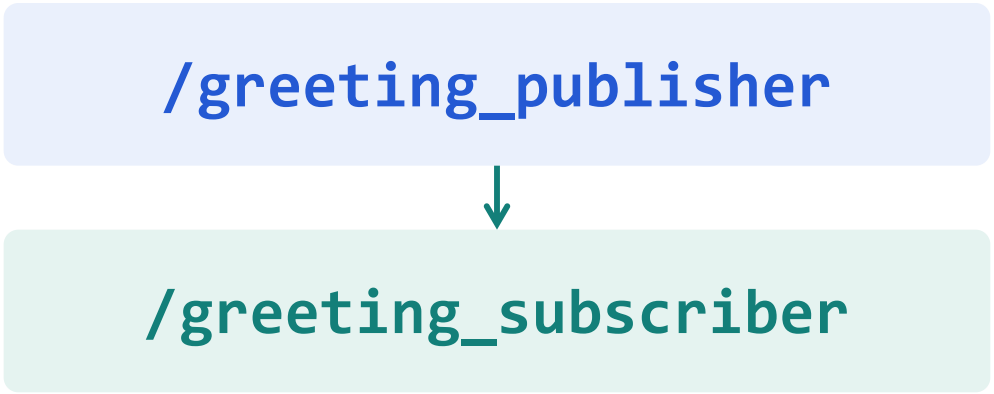
DDS discovery · QoS · lifecycle

ROS 2 = generation. Jazzy = distribution.

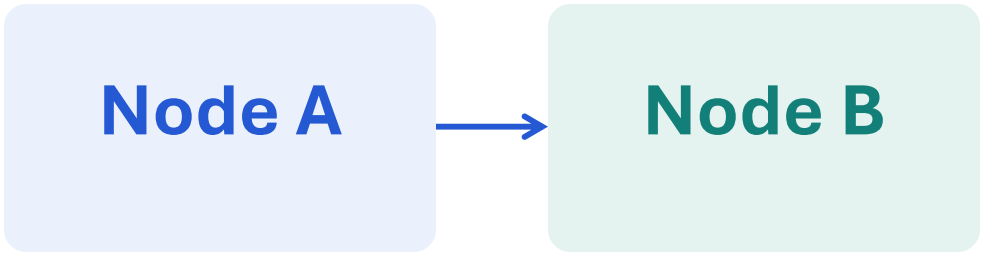
Nodes and the ROS graph

A node has a responsibility. A process hosts running code.

THIS DEMO: TWO PROCESSES



ALSO VALID: ONE PROCESS



The graph describes nodes and their communication connections.

Topics carry typed messages

Publishers send. Subscribers receive. One topic can connect many.



String interface

```
string data
```

```
message = String()
message.data = 'Hello, robotics!'
```

Choose the right interface

Match the communication pattern to the task.

Topic

data → data → data

Sensor readings

Service

request → response

Reset odometry

Action

goal → feedback → result

Navigate + cancel

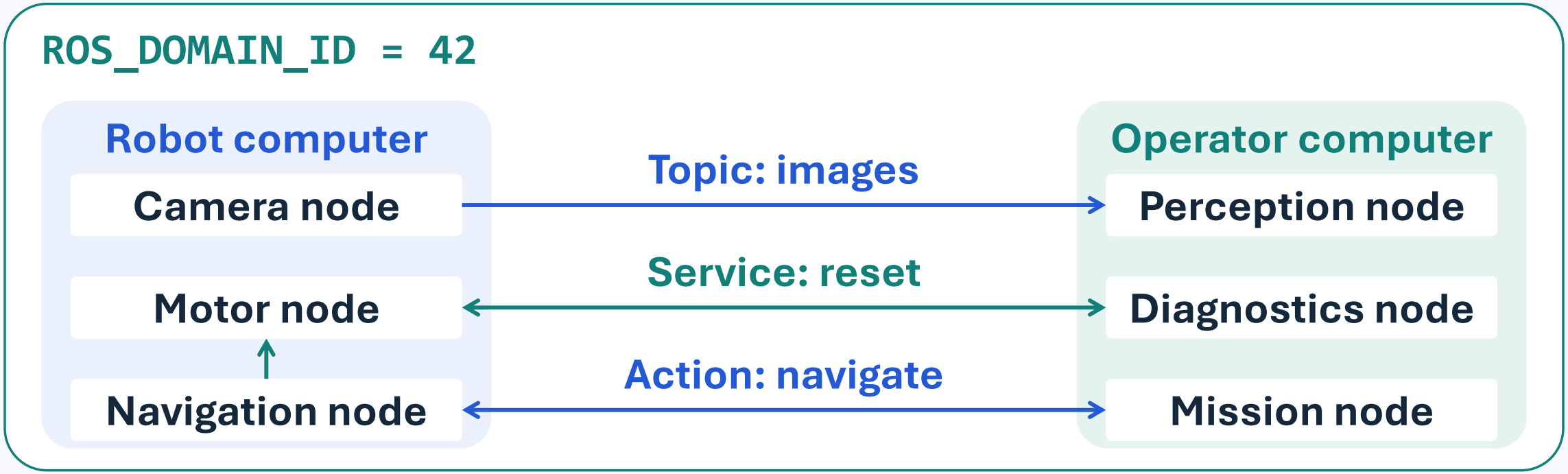
Parameter

node configuration

Exposure / frame name

One graph, multiple computers

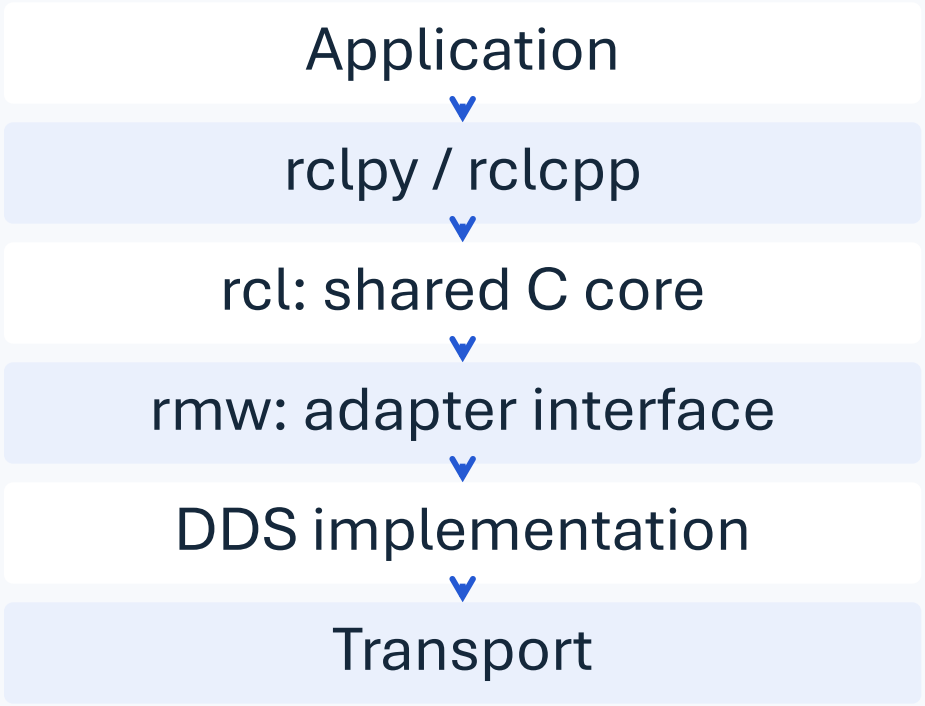
Nodes communicate locally or across computers in one ROS domain.

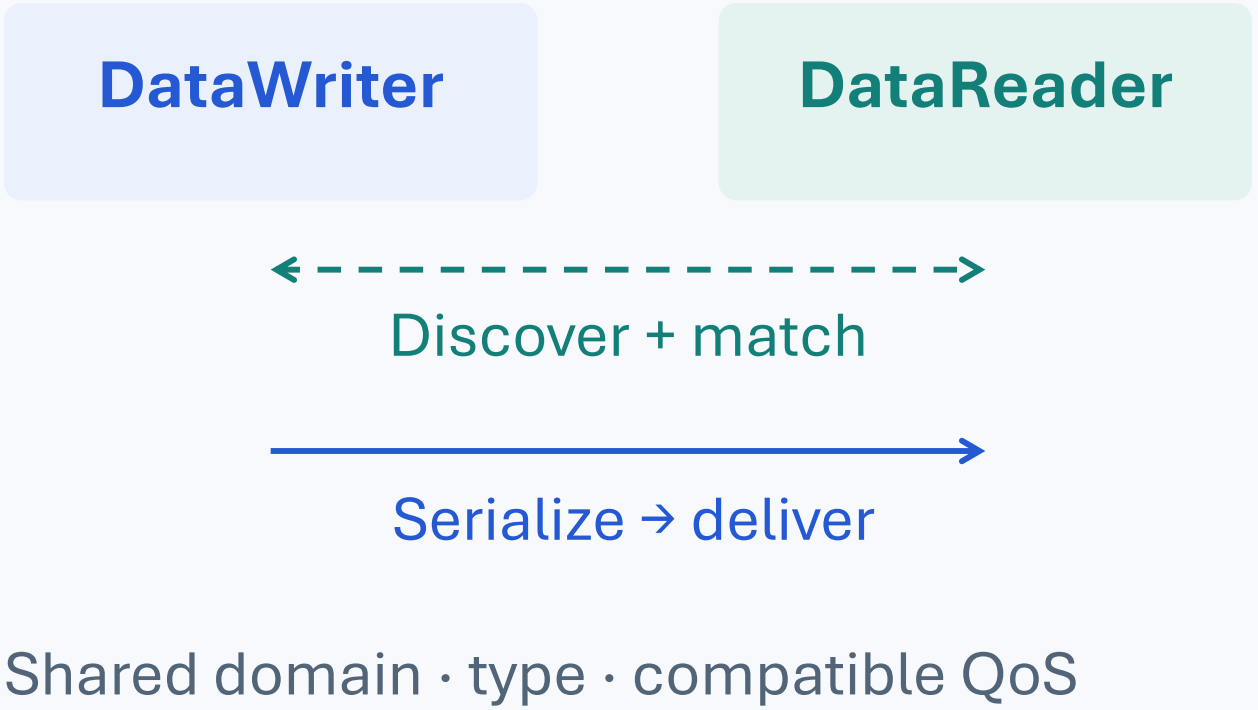


Same domain + reachable network + compatible interfaces.

ROS 2 architecture and DDS

DDS = Data Distribution Service. RTPS = its wire protocol.





QoS shapes delivery

Quality of Service sets the contract between endpoints.

Reliability

RELIABLE

Retry lost samples

Durability

VOLATILE

New data after joining

History

KEEP_LAST: 10

Keep 10 samples

10 = history depth

1.0 = timer interval (s)

A restarted subscriber does not replay missed greetings.

Execution and lifecycle

DDS delivers data. The executor runs your callbacks.

ORDINARY NODE: OUR DEMO

init → construct node

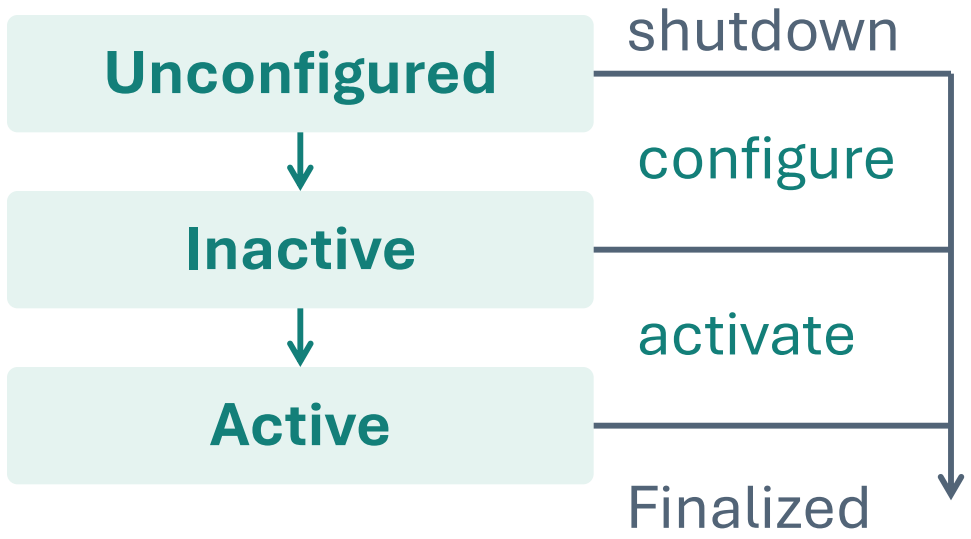
↓

spin → ready callbacks

↓

Ctrl+C → cleanup

MANAGED NODE: OPTIONAL



Inspect before debugging

Source Jazzy in every terminal. Run these scripts directly.

`ros2 node list`	**Find nodes**
`ros2 topic list`	**Find topics**
`ros2 topic info /chatter --verbose`	**Type + QoS**
`ros2 interface show std_msgs/msg/String`	**Read fields**
`ros2 topic echo /chatter`	**Observe data**

topic echo becomes another subscriber.

Let's read it. Then run it.

Predict the behavior before changing the running system.



`std_msgs/msg/String`

01

Subscriber stops?

02

Subscriber restarts?

03

Publisher restarts?

OPTIONAL / DEEPER DISCUSSION

DDS discovery and RTPS

Find participants. Match endpoints. Exchange application data.

Participants → **Endpoint discovery** → **Application data**

SPDP → SEDP → matching writers and readers

Reliable delivery can recover missing samples.

Sequence → heartbeat → ACK/NACK → retransmission

OPTIONAL / DEEPER DISCUSSION

QoS compatibility

The publisher must satisfy the subscriber's requested policy.

Publisher offers	Reader requests	Compatible?
Best effort	Best effort	Yes
Best effort	Reliable	No
Reliable	Best effort	Yes
Reliable	Reliable	Yes

Reliability shown here; other policies must also be compatible.

OPTIONAL / DEEPER DISCUSSION

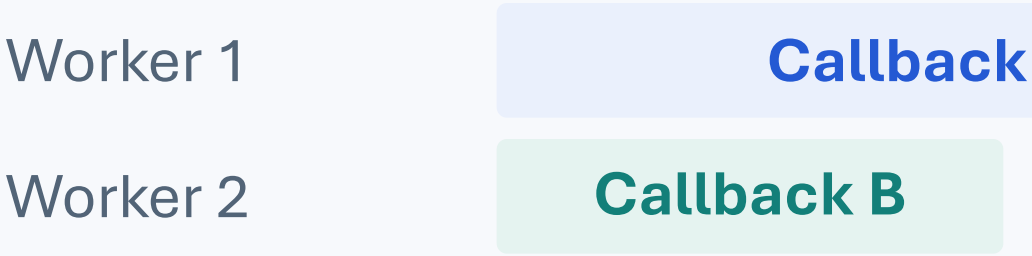
Executors and concurrency

Threads help only when callback groups allow work to overlap.

Single-threaded: one callback at a time



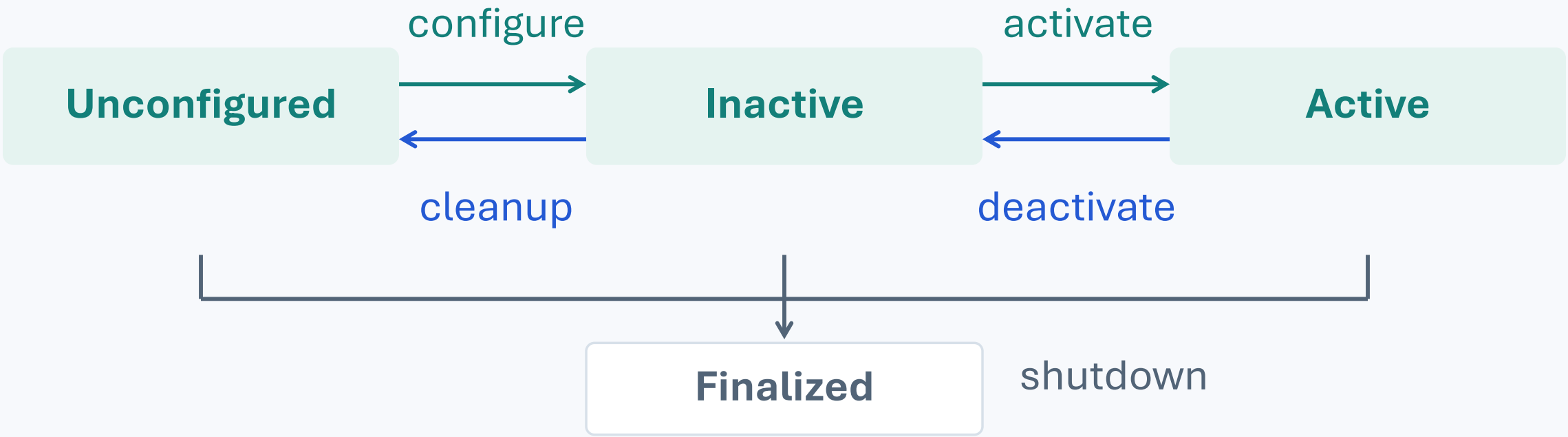
Multithreaded: eligible callbacks can overlap



OPTIONAL / DEEPER DISCUSSION

Managed lifecycle transitions

A supervisor requests transitions; callbacks manage resources.



Callbacks: success / failure; errors → ErrorProcessing

OPTIONAL / DEEPER DISCUSSION

Migrating from ROS 1

Keep the architectural ideas. Port the implementation.



APIs · build · parameters · launch · QoS

Bridge support depends on interfaces, distributions, and operating systems.